

Arduino IDE - MPU6050 V1.1

Date: 18/11/2025

Tags: [PEE](#), [Code](#), [Microcontroller](#), [ESP32](#), [Algorithms](#)

Description

This page main objective is documenting and explaining my progress with the code for the ESP32 to work only with our IMU, MPU6050.

The version 1.1 adds the calibration of the raw data by subtracting the offset and then converting the raw data to known units.

Code

- By passing the previous version of the code ([Arduino IDE - MPU6050 V1.0](#)) to MATLAB via serial interface () and doing the average of all data arrays I obtained the necessary offsets.

Defining constant for Offsets

```
1  const float ACCEL_SENSITIVITY = 16384.0; // LSB/g (para a escala de ±2g)
2  const float GYRO_SENSITIVITY = 131.0;    // LSB/°/s (para a escala de ±250°/s)
3
4  int16_t AcX_offset = -398;
5  int16_t AcY_offset = -127;
6  int16_t AcZ_offset = 383;    // (16767 - 16384)
7  int16_t GyX_offset = -344;
8  int16_t GyY_offset = -50;
9  int16_t GyZ_offset = -9;
```

- `const float ACCEL_SENSITIVITY = 16384.0` In the datasheet the accelerometer and the gyroscope have **sensitivity per LSB**. In this case LSB is "*Least Significant Bits*". The data that comes from any digital sensor or any data via digital comes as counts or LSB. In the datasheet we can say that the LSB sensitivity has the following units $LSB/(^{\circ}/s)$ or LSB/g . Where the $^{\circ}$ means degrees and g means the gravitational acceleration ($g = 9.81 m^2/s$). This constant will be used to transform the digital raw data into real data like degrees or acceleration.
- `int16_t AcX_offset = -398;` The values on this constant are product of averaging all of data in an array of a simulation that gave 3346 samples.
- `int16_t AcZ_offset = 383 // (16767 - 16384)` The value 16767 is given by the average of the array but the 16384 is given by the sensitivity. Why you may ask. In our test the MPU-6050 has the Z axis pointing down, as the accelerometer measures all linear acceleration, this axis will always measure the gravitational acceleration.

Calibrating Values

```
1  int16_t AcX_cal = AcX_raw - AcX_offset;
2  int16_t AcY_cal = AcY_raw - AcY_offset;
3  int16_t AcZ_cal = AcZ_raw - AcZ_offset;
```

```

4      int16_t GyX_cal = GyX_raw - GyX_offset;
5      int16_t GyY_cal = GyY_raw - GyY_offset;
6      int16_t GyZ_cal = GyZ_raw - GyZ_offset;
7
8      ax = ((float)AcX_cal / ACCEL_SENSITIVITY) * 9.81;
9      ay = ((float)AcY_cal / ACCEL_SENSITIVITY) * 9.81;
10     az = ((float)AcZ_cal / ACCEL_SENSITIVITY) * 9.81;
11
12     gx = (float)GyX_cal / GYRO_SENSITIVITY;
13     gy = (float)GyY_cal / GYRO_SENSITIVITY;
14     gz = (float)GyZ_cal / GYRO_SENSITIVITY;
15

```

- `int16_t AcX_cal = AcX_raw - AcX_offset;` This line and the similar ones subtract each offset in each axis of the MPU-6050.
- `ax = ((float) AcX_cal / ACCEL_SENSITIVITY) * 9.81;` This line takes the calibrated acceleration and multiplies by the gravitational acceleration to ax be in m^2/s . This line makes AcX_cal into a float value.
- `gx = (float)GyX_cal / GYRO_SENSITIVITY;` This line takes the calibrated angles and divides by the sensitivity to gx be in $^{\circ}$. This line makes GyX_cal into a float value.